

Lista de Exercícios #6 – Métodos de Busca - João Comini Cesar de Andrade

Questão 01. Sobre os principais conceitos de métodos de busca, responda:

1. Quais são os cinco componentes exigidos para definir um problema de busca em IA? Defina e ilustre cada um com o exemplo do 8-puzzle.

R: Os componentes exigidos são:

- **Estado inicial:** a situação em que o agente começa (ex: qualquer configuração inicial das peças do tabuleiro).
- **Conjunto de ações:** movimentos disponíveis para o agente (ex: mover o quadrado vazio para a esquerda, direita, cima ou baixo).
- **Modelo de transição:** função sucessora com os resultados das ações (ex: o novo posicionamento das peças após um movimento).
- **Teste do objetivo:** estado ou propriedade abstrata a ser atingida (ex: verificar se as peças estão na configuração do estado objetivo previamente especificada).
- **Custo da solução:** a soma do custo dos passos individuais (ex: cada ação ou movimento de peça tem custo de valor 1).

2. Compare os algoritmos de Busca em Largura (BFS) e Busca em Profundidade (DFS) nos critérios de completude e otimalidade. Descreva um cenário ou problema em que a Busca em Profundidade poderia falhar (não terminar).

R: A Busca em Largura (BFS) é considerada um algoritmo completo (desde que a profundidade seja finita) e também ótimo (desde que o custo seja uma função não decrescente da profundidade). A Busca em Profundidade (DFS), por outro lado, não é completa e também não é ótima. Um cenário em que a DFS pode falhar é em um espaço de busca infinito; como a busca vai sempre tentando descer no nível mais profundo, o algoritmo pode se perder e entrar em um caminho infinito, nunca encontrando um nó final objetivo.

3. A Busca de Custo Uniforme (equivalente ao algoritmo de Dijkstra) é uma generalização da BFS para grafos com custos variáveis nas arestas. Explique por que a BFS simples não é adequada quando as ações têm custos diferentes. Descreva como a Busca de Custo Uniforme garante encontrar a solução de menor custo. Qual estrutura de dados é fundamental para sua eficiência?

R: A BFS simples não é adequada porque ela trabalha expandindo apenas o nó mais raso de forma uniforme por nível da árvore, não sendo otimizada caso os passos no grafo tenham custos variados (ela funciona como algo "ótimo" somente quando os custos forem todos

iguais). A Busca de Custo Uniforme corrige isso e garante a solução de menor custo ao priorizar e sempre expandir o nó que possui o caminho de menor custo acumulado até o momento. A estrutura de dados que é fundamental para ordenar e trazer essa eficiência é uma fila de prioridades, também chamada de *heap*.

4. O que é uma função heurística $h(n)$ e como ela diferencia a Busca com Informação da Busca sem Informação? Cite pelo menos um exemplo de heurística abordada no material (por exemplo, no mapa da Romênia ou no 8-puzzle).

R: Uma função heurística $h(n)$ fornece uma indicação matemática de qual é o melhor caminho até a solução desejada, sendo definida comumente como o custo estimado do melhor caminho de um nó n até o objetivo. Ela difere e separa as classes de algoritmos porque fornece conhecimento para que a Busca com Informação saiba se um estado é "mais promissor", enquanto a Busca sem Informação é executada de forma "cega", não possuindo conhecimentos sobre o ambiente além do que foi estritamente formulado no problema inicial. Um exemplo clássico no material é adotar a distância em linha reta das cidades até o destino final como heurística principal para viagens (como no mapa da Romênia até Bucareste).

5. Tanto a Busca Gulosa quanto o A^* utilizam funções heurísticas, mas com estratégias diferentes. Compare os dois algoritmos quanto às suas funções de avaliação $f(n)$ e demonstre com um exemplo (como o problema da viagem na Romênia) por que a Busca Gulosa pode encontrar soluções subótimas enquanto o A^* , com heurística admissível, sempre encontra a solução ótima.

R: Na estratégia da Busca Gulosa, a função de avaliação obedece apenas a $f(n) = h(n)$, visando reduzir sempre e exclusivamente o custo imediato de expansão. No Algoritmo A^* , a função tem a intenção de minimizar o custo total combinando a heurística e o custo de trajeto real já percorrido, sendo formatada como $f(n) = g(n) + h(n)$. A Busca Gulosa muitas vezes encontra caminhos subótimos porque pode escolher ramificar para um nó puramente por ele possuir a menor estimativa $h(n)$ direta para o fim (como por exemplo pegar uma cidade "geograficamente perto"), mesmo sem se preocupar que o caminho total que ele traçou até chegar ali possa ser gigantesco e ineficiente. O algoritmo A^* , por outro lado, usa esse passado $g(n)$ nos cálculos; contanto que ele utilize uma heurística "admissível" (que obedece à regra de nunca superestimar o custo de atingir o objetivo), ele possuirá garantias matemáticas de otimalidade e completude no caminho encontrado.

6. Você está desenvolvendo um agente inteligente para três cenários distintos: (a) encontrar o caminho mais curto em um mapa de cidades onde as distâncias em linha reta são conhecidas; (b) jogar xadrez contra um oponente humano com limite de tempo por jogada; (c) resolver um labirinto desconhecido sem nenhuma informação prévia sobre o ambiente. Para cada cenário, indique qual estratégia de busca seria mais adequada e justifique sua escolha com base nas propriedades de cada algoritmo estudado.

R: As alocações podem ser propostas como as seguintes:

- **Cenário (a) - Mapa de cidades:** A estratégia mais adequada é o **Algoritmo A^*** . Ao possuímos informações métricas das distâncias reais ($g(n)$) além de uma estimativa disponível em linha reta (que atua perfeitamente como heurística admissível, $h(n)$), o A^* minimiza esse custo total e fornece como garantia final e comprovada o trajeto de solução ótimo.
- **Cenário (b) - Jogar xadrez:** A estratégia mais adequada é o **MINIMAX com Poda Alfa-Beta**. Partidas de xadrez são ambientes que precisam ser visualizados como jogos de soma zero, logo a lógica base tem que ser tomada contra um oponente oposto. A grande aplicação da poda alfa-beta é fundamental no cenário por causa da estipulação do limite de tempo: a técnica irá desconsiderar agressivamente ramos inteiros que são inúteis debaixo de escolhas adversas que o "MIN" não faria de qualquer forma, diminuindo o processamento exponencial original de forma muito mais rápida para encontrar o veredito da jogada a tempo.
- **Cenário (c) - Labirinto desconhecido:** A estratégia mais adequada é a **Busca com Aprofundamento Iterativo**. A falta de qualquer informação prévia impede qualquer método heurístico. Lançar o DFS base direto de forma cega cria o perigo de caminhos infinitos. O aprofundamento iterativo mitiga todos esses problemas limitando e efetuando várias buscas controladas sucessivamente de profundidade, mesclando as vantagens sem necessitar abarrotar o espaço na memória.

Questão 02. Considere o problema de busca definido pelo seguinte grafo, onde A é o estado inicial e G é o estado objetivo, e o custo de cada ação é indicado na aresta correspondente.

(Nota: Com base nos dados extraídos do documento, o grafo reconstruído possui as seguintes conexões e custos: A-B(2), A-C(4), B-C(1), B-E(5), B-F(6), C-E(5), E-F(3), E-H(1), F-G(2), H-G(2). As heurísticas são: $h(A)=5$, $h(B)=2$, $h(C)=2$, $h(E)=2$, $h(F)=100$, $h(G)=0$, $h(H)=1$).

Considerando a ordem de visitação alfabética para desempate, abaixo estão os caminhos e custos encontrados por cada algoritmo:

1. Busca em Largura (BFS)

A BFS explora o grafo nível por nível (nós mais rasos primeiro).

- **Árvore/Ordem:** $A \rightarrow (B, C) \rightarrow$ a partir de B expande (E, F) $\rightarrow$ a partir de F expande (G). O objetivo G é alcançado no nível 3.
- **Caminho final:** $A \rightarrow B \rightarrow F \rightarrow G$
- **Custo da solução:** $2 + 6 + 2 = 10$

2. Busca em Profundidade (DFS)

A DFS explora o ramo mais profundo possível antes de retroceder. Priorizando a ordem alfabética na expansão (A vai para B primeiro, B vai para C primeiro, etc.).

- **Árvore/Ordem:** $A \rightarrow B \rightarrow C \rightarrow E \rightarrow F \rightarrow G$ (encontra o objetivo e para).

- **Caminho final:** $A \rightarrow B \rightarrow C \rightarrow E \rightarrow F \rightarrow G$
- **Custo da solução:** $2 + 1 + 5 + 3 + 2 = 13$

3. Busca de Custo Uniforme (UCS)

Expande sempre o nó com o menor custo acumulado do caminho percorrido até o momento, $g(n)$.

- **Árvore/Ordem de expansão:** $A(0) \rightarrow B(2) \rightarrow C(3) \rightarrow C(4) \rightarrow E(7) \rightarrow H(8) \rightarrow F(8) \rightarrow G(10)$.
- **Caminho final:** $A \rightarrow B \rightarrow E \rightarrow H \rightarrow G$ (Nota: $A \rightarrow B \rightarrow F \rightarrow G$ também possui custo 10, mas E precede F na ordem alfabética).
- **Custo da solução:** $2 + 5 + 1 + 2 = 10$

4. Busca Gulosa

Expande sempre o nó que parece estar mais próximo do objetivo, baseando-se apenas na heurística: $f(n) = h(n)$.

- **Árvore/Ordem de expansão:** $A(5) \rightarrow B(2)$ (empate com C , escolhe B) $\rightarrow C(2) \rightarrow E(2) \rightarrow H(1) \rightarrow G(0)$.
- **Caminho final:** $A \rightarrow B \rightarrow C \rightarrow E \rightarrow H \rightarrow G$
- **Custo da solução:** $2 + 1 + 5 + 1 + 2 = 11$ (encontrou um caminho subótimo).

5. Algoritmo A^* (A-Estrela)

Expande minimizando o custo total estimado, combinando o custo real percorrido e a heurística: $f(n) = g(n) + h(n)$.

- **Árvore/Ordem de expansão:** * $A: 0 + 5 = 5$
 - $B: 2 + 2 = 4$ (escolhido)
 - C (via B): $3 + 2 = 5$ (escolhido)
 - E (via B): $7 + 2 = 9$ (escolhido)
 - H (via E): $8 + 1 = 9$ (escolhido)
 - G (via H): $10 + 0 = 10$ (objetivo atingido!)
- **Caminho final:** $A \rightarrow B \rightarrow E \rightarrow H \rightarrow G$
- **Custo da solução:** $2 + 5 + 1 + 2 = 10$

Questão 03. Considere a seguinte árvore representando um jogo de soma zero, onde os triângulos apontando para cima são nós MAX, os triângulos apontando para baixo são nós MIN e os quadrados são nós objetivos (terminais) com o valor correspondente da função de utilidade para o jogador MAX. Aplique o algoritmo MINIMAX para encontrar a melhor ação para o jogador MAX na raiz. Mostre a árvore indicando o valor minimax para cada nó.

Neste problema, aplicamos o algoritmo **MINIMAX**, que funciona de baixo para cima (das folhas para a raiz). O jogador MAX quer maximizar o valor, e o jogador MIN quer minimizar.

1. Avaliação do Nível dos Nós Terminais (Quadrados):

Os valores da utilidade fornecidos na imagem, da esquerda para a direita, são:

- Subárvore esquerda: 5, 2, 3
- Subárvore central: 1, 2, 3
- Subárvore direita: 4, 0, 5

2. Avaliação do Nível MIN (Triângulos para baixo):

Cada nó MIN escolherá o **menor** valor possível entre seus filhos terminais

- **Nó MIN 1 (Esquerda):** $\min(5, 2, 3) = 2$
- **Nó MIN 2 (Centro):** $\min(1, 2, 3) = 1$
- **Nó MIN 3 (Direita):** $\min(4, 0, 5) = 0$

3. Avaliação do Nível MAX / Raiz (Triângulo para cima):

O nó MAX na raiz escolherá o **maior** valor possível entre as opções deixadas pelos nós MIN.

- **Nó MAX (Raiz):** $\max(2, 1, 0) = 2$

Resposta Final:

- O valor Minimax para o nó raiz é **2**.
- A melhor ação para o jogador MAX na raiz é escolher o **caminho da esquerda** (que leva ao nó MIN que retorna o valor 2).